

1. Codificación y Base de Datos

Repositorio de Código Fuente

El código fuente de la plataforma **Healthcare ETL Platform** está alojado en GitHub y es completamente funcional.

- **URL del Repositorio:** <https://github.com/ivanch1-23/healthcare>
- **Rama principal:** `main`
- **Último commit:** `42cf770` - Initial commit: Healthcare ETL Platform

Estructura del Repositorio

```
healthcare-etl-platform/
├── backend/                # Aplicación Django (Backend)
│   ├── config/            # Configuración del proyecto Django
│   │   ├── settings.py    # Configuración general, BD, apps, middleware
│   │   ├── urls.py        # Enrutamiento global de URLs
│   │   ├── wsgi.py / asgi.py # Puntos de entrada WSGI/ASGI
│   └── apps/              # Módulos funcionales
│       ├── authentication/ # Autenticación y roles (JWT)
│       ├── etl/            # Motor ETL (Extract, Transform, Load)
│       ├── analytics/      # Estadísticas y KPIs médicos
│       ├── ml/             # Machine Learning (Random Forest)
│       ├── dashboard/      # Vistas del frontend
│       └── reports/        # Exportación de reportes
│   ├── manage.py          # Comando principal de Django
│   ├── requirements.txt    # Dependencias Python
│   ├── Dockerfile         # Imagen Docker del backend
│   ├── populate.py        # Script de inicialización
│   └── verify_etl.py       # Script de verificación ETL
├── frontend/              # Frontend (Templates + Static)
│   ├── templates/         # Plantillas HTML (Django Templates)
│   │   ├── base.html      # Layout base con navbar y lógica JWT
│   │   ├── login.html     # Pagina de inicio de sesión
│   │   ├── dashboard.html  # Dashboard analítico con KPIs y graficas
│   │   ├── etl.html       # Panel de control ETL
│   │   ├── pacientes.html  # Directorio maestro de paciente
│   │   ├── ml.html        # Panel de Machine Learning
│   │   ├── reportes.html   # Reportes y exportación
│   │   └── usuarios.html   # Gestión de usuarios (Admin)
│   └── static/
│       ├── css/custom.css  # Estilos personalizados
│       └── datasets/       # Dataset de prueba (Excel con 1800 registros)
├── Entregables/           # Documentación del proyecto
├── docker-compose.yml     # Orquestación Docker
├── database_schema.sql     # Esquema SQL para PostgreSQL
└── .env.example           # Variables de entorno de ejemplo
```

Tecnologías Utilizadas

Componente	Tecnología
------------	------------

Backend Framework	Django 4.2.30 + Django REST Framework 3.17.1
Autenticación	JWT (djangorestframework-simplejwt)
Base de Datos	SQLite (desarrollo) / PostgreSQL 15 (producción)
Frontend	HTML5, CSS3, Bootstrap 5, Chart.js
ETL	Pandas 3.0 + OpenPyXL
Machine Learning	Scikit-learn 1.9 (Random Forest Classifier)
Documentación API	drf-spectacular (Swagger UI)
Contenedores	Docker + Docker Compose

Instrucciones de Instalación y Ejecución

Requisitos previos: Python 3.9+, pip

```
# 1. Clonar el repositorio
git clone https://github.com/ivanchi-23/healthcare.git
cd healthcare/healthcare-eti-platform

# 2. Crear y activar entorno virtual
python -m venv .venv
.venv\Scripts\activate # Windows
# source .venv/bin/activate # Linux/Mac

# 3. Instalar dependencias
cd backend
pip install -r requirements.txt

# 4. Configurar variables de entorno
# Copiar .env.example a .env en la raíz del proyecto y ajustar si es necesario

# 5. Ejecutar migraciones
python manage.py migrate

# 6. Inicializar base de datos con datos semilla (crea admin, ejecuta ETL y entrena ML)
python manage.py seed

# 7. Iniciar servidor
python manage.py runserver 0.0.0.0:8000
```

Acceso: <http://localhost:8000>
Documentación API: <http://localhost:8000/api/docs/>
Admin Django: <http://localhost:8000/admin/>

Script SQL de la Base de Datos

Esquema para PostgreSQL (Producción)

Archivo: backend/database_schema.sql

```

-- 1. Tabla de Usuarios Personalizados (authentication_customuser)
CREATE TABLE IF NOT EXISTS authentication_customuser (
    id BIGSERIAL PRIMARY KEY,
    password VARCHAR(128) NOT NULL,
    last_login TIMESTAMP WITH TIME ZONE NULL,
    is_superuser BOOLEAN NOT NULL DEFAULT FALSE,
    username VARCHAR(150) UNIQUE NOT NULL,
    first_name VARCHAR(150) NOT NULL,
    last_name VARCHAR(150) NOT NULL,
    email VARCHAR(254) NOT NULL,
    is_staff BOOLEAN NOT NULL DEFAULT FALSE,
    is_active BOOLEAN NOT NULL DEFAULT FALSE,
    date_joined TIMESTAMP WITH TIME ZONE NOT NULL,
    rol VARCHAR(20) NOT NULL DEFAULT 'analista'
);

-- 2. Tabla de Pacientes Clinicos (etl_pacienteclinico)
CREATE TABLE IF NOT EXISTS etl_pacienteclinico (
    id BIGSERIAL PRIMARY KEY,
    id_paciente INTEGER UNIQUE NOT NULL,
    nombres VARCHAR(150) NOT NULL,
    apellidos VARCHAR(150) NOT NULL,
    edad INTEGER NOT NULL,
    sexo VARCHAR(20) NOT NULL,
    peso DOUBLE PRECISION NOT NULL,
    altura DOUBLE PRECISION NOT NULL,
    "IMC" DOUBLE PRECISION NOT NULL,
    "presión_sistólica" INTEGER NOT NULL,
    "presión_diastólica" INTEGER NOT NULL,
    frecuencia_cardiaca INTEGER NOT NULL,
    glucosa DOUBLE PRECISION NOT NULL,
    colesterol DOUBLE PRECISION NOT NULL,
    "saturación_oxígeno" DOUBLE PRECISION NOT NULL,
    temperatura DOUBLE PRECISION NOT NULL,
    antecedentes_familiares BOOLEAN NOT NULL,
    fumador BOOLEAN NOT NULL,
    consumo_alcohol BOOLEAN NOT NULL,
    "actividad_física" VARCHAR(100) NOT NULL,
    "diagnostico_preliminar" VARCHAR(250) NOT NULL,
    riesgo_enfermedad VARCHAR(50) NOT NULL,
    fecha_consulta DATE NOT NULL
);

-- 3. Tabla de Registros ETL (etl_registroetl)
CREATE TABLE IF NOT EXISTS etl_registroetl (
    id BIGSERIAL PRIMARY KEY,
    fecha TIMESTAMP WITH TIME ZONE NOT NULL,
    registros_procesados INTEGER NOT NULL,
    registros_limpios INTEGER NOT NULL,
    duplicados_eliminados INTEGER NOT NULL,
    tiempo_ejecucion DOUBLE PRECISION NOT NULL,
    estado VARCHAR(20) NOT NULL,
    errores TEXT NULL,
    usuario_fk_id BIGINT NOT NULL
    REFERENCES authentication_customuser (id)
    ON DELETE CASCADE DEFERRABLE INITIALLY DEFERRED
);

```

Esquema para SQLite (Desarrollo)

Archivo: backend/entregables_script.sql

Contiene la estructura DDL completa exportada desde SQLite, incluyendo todas las tablas del sistema:

- `authentication_customuser` - Usuarios con roles (administrador, médico, analista)
- `etl_pacienteclinico` - Registros clínicos de pacientes (23 campos)
- `etl_registroetl` - Historial de ejecuciones del pipeline ETL
- `django_migrations`, `django_content_type`, `auth_permission`, `auth_group`, `django_session` - Tablas del framework Django

Modelo de Datos (ORM Django)

Modelo CustomUser (`apps/authentication/models.py`):

- Extiende `AbstractUser` de Django
- Campo adicional: `rol` (administrador, médico, analista)

Modelo PacienteClinico (`apps/etl/models.py`):

- 23 campos que capturan información clínica completa
- Campos clave: `id_paciente` (único), `IMC`, `glucosa`, `presion_sistolica`, `riesgo_enfermedad`, `diagnostico_preliminar`

Modelo RegistroETL (`apps/etl/models.py`):

- Bitácora de cada ejecución del pipeline ETL
- Campos: `fecha`, `usuario_fk`, `registros_procesados`, `registros_limpios`, `duplicados_eliminados`, `tiempo_ejecucion`, `estado`, `errores`